

Lec 04 - More Higher-Order Functions

COMPSCI 220 - Perretta

Objectives

- Practice **implementing and using** higher-order functions
- Practice searching **documentation** to find the right tools
- Practice selecting the **right HOF for the job**
 - or writing a custom one if what you need doesn't already exist

Example 1: every

Write a generic function `every` that generalizes the structure of `allStringsLowercase` below.

```
function allStringsLowercase(strs: string[]) {  
  for (const str of strs) {  
    if (str !== str.toLowerCase()) {  
      return false;  
    }  
  }  
  return true;  
}
```

Extra Practice: some

Write a generic function `some` that behaves as described

```
// Takes in an array of elements of the same type  
// and a unary predicate.  
// Returns true if the predicate evaluates to true for  
// at least one element, false otherwise.
```

Example 2: Using Built-In HOFs

Everything you need for this exercise is in the **MDN Array documentation**. Do not use search engines, LLMs or other resources.

Write a function that takes in a **2D array of Snowfall objects**. Assume each row contains entries from a single month. Assume entries are sorted by date, ascending.

Return an array with the first date of each month on which there was at least 2 inches of snow. If there are no such days in a month, the returned array should contain an empty string for that month instead.

```
class Snowfall {  
  constructor(public date: string,  
              public inches: number) {}  
}  
  
const dataByMonth = [  
  [new Snowfall('2026-01-14', 2),  
   new Snowfall('2026-01-18', 1)],  
  [new Snowfall('2026-02-01', 1),  
   new Snowfall('2026-02-27', 3)],  
  [new Snowfall('2026-03-08', 1)],  
];  
  
// Result  
// [ '2026-01-14', '2026-02-27', '' ]
```

Extra Practice: Supplementing Built-In HOFs

Write a function that takes in an array of Snowfall objects. The entries are **not assumed** to be sorted by date.

```
const dataUnsorted = [  
  new Snowfall("2026-01-21", 3),  
  new Snowfall("2026-01-12", 1),  
  new Snowfall("2026-01-15", 2.5),  
];
```

Return the first date on which there was at least 2 inches of snow. If there are no such days, return an empty string.

The function should run in $O(n)$ time. I.e., do not sort the array.

Hint: `string.localeCompare` can be used to check which of two date strings is earlier.

Your solution must **not use loops**. Implement and use a new higher-order-function that lends itself to an expressive, understandable solution. The new HOF should be general enough to be usable with any array type. (The new HOF may use loops).

Reduce

- General-purpose HOF for **accumulating** values

```
function sum(  
  values: number[]  
): number {  
  let result = 0;  
  for (const val of values) {  
    result += val;  
  }  
  return result;  
}  
console.log(sum([42, 43, 44]));
```

129

```
function flatten<T>(  
  arrays: T[][]  
): T[] {  
  let result: T[] = [];  
  for (const array of arrays) {  
    result = result.concat(array);  
  }  
  return result;  
}  
console.log(  
  flatten([["egg"], ["bacon", "spam"]])  
);
```

["egg", "bacon", "spam"]

Reduce

```
function sum(  
  values: number[]  
): number {  
  let result = 0;  
  for (const val of values) {  
    result += val;  
  }  
  return result;  
}
```

```
type AccumFuncType<ElementType, ResultType>  
  = (soFar: ResultType,  
     currentValue: ElementType) => ResultType;  
  
function reduce<ElementType, ResultType>(  
  array: ElementType[],  
  accumFunc: AccumFuncType<ElementType, ResultType>,  
  initialValue: ResultType,  
): ResultType {  
  let result = initialValue;  
  for (const item of array) {  
    result = accumFunc(result, item);  
  }  
  return result;  
}
```

When to Use Reduce

- OK for simple accumulations, e.g., sum

```
const nums = [1, 2, 3];  
console.log(nums.reduce((soFar, currentValue) => soFar + currentValue));
```

6

- Better to define a function `sum` and use it directly

```
// Loop vs. reduce for implementation doesn't really matter  
function sum(nums: number[]): number { ... }  
  
console.log(sum([1, 2, 3]));
```

When to Use Reduce

- If a more expressive option exists, prefer that over reduce

Less expressive:

```
const nums = [1.2, 2.5444, 3];  
console.log(  
  nums.reduce(  
    (soFar, currentValue) =>  
      soFar + " "  
      + currentValue.toFixed(2),  
    "",  
  ),  
);
```

1.20 2.54 3.00

More expressive:

```
const nums = [1.2, 2.5444, 3];  
console.log(  
  nums.map(num) => num.toFixed(2))  
  .join(" ");
```

1.20 2.54 3.00